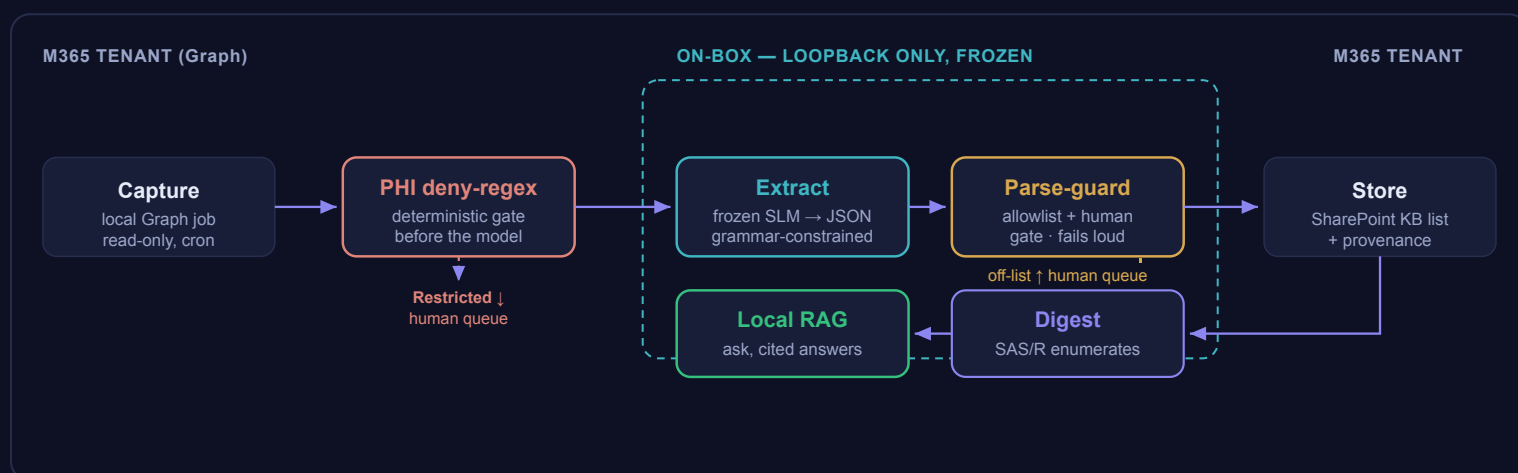


Study Operations KB — the Copilot-free, local-SLM path

⚠ **PROPOSED** — pending management approval, not in production today. This describes a **proposed future-state** operating layer, not how we work now. Today's deliverables run through validated tools + our in-house team; every AI output is human-signed. New here? Start with [the onboarding page](#).

This builds the **same self-updating study-operations knowledge base** as the Biostatistics × Copilot wiki — a study email becomes a structured, queryable entry, and the team can ask the knowledge base questions — but the AI step runs on an **on-prem, frozen Small Language Model (SLM)**, and it uses **no M365 Copilot, no Copilot Studio, no AI Builder**, end to end.



The full path: a **local Graph capture job** → a **deterministic PHI deny-regex** (Restricted mail drops to a human queue before the model) → a **frozen on-prem SLM** extract to strict JSON → an allowlist + **human-gated parse-guard** → the **SharePoint KB**, which feeds the **local RAG** and the **deterministic digest**. Only the AI step is on-box; the model never produces a number.

The one rule (unchanged). This is the *operational / knowledge layer only*. The model classifies, extracts, routes, and drafts short grounded text — it **never produces a reported number, never reconciles, never decides**. Validated tools (Phoenix WinNonlin, Pinnacle 21, the validated SAS/R pipeline) and a qualified human own every regulated value. The KB carries a visible “informational ops support — not a source of record” disclaimer. Never route PHI / unblinded / participant-level data.

Read these three honesty points before anything else

1 · Local does not mean reliable. A small (7–8B), quantized model *hallucinates* — and *hallucinates MORE than a frontier model*, and is more prompt/format-sensitive. Running on-prem buys **data sovereignty and reproducibility, not correctness**. The trustworthiness of this pipeline comes from the **engineering discipline you control** — schema-constrained decoding, an allowlist validator, RAG grounding, and a **human-gated parse-guard** — not from the model being local.

2 · The real reason to go local for this task is the IT & reproducibility path — not that Copilot can't do it. For a de-identified ops/knowledge/drafting layer, M365 Copilot is genuinely capable. You go Copilot-free here to (a) **remove the Copilot-specific IT frictions** (the DLP-connector cross-group block, the 01-Nov-2026 AI-Builder

credit retirement, Copilot Studio per-use licensing + the tenant agent-publish gate, cloud egress, and the no-model-freeze reproducibility gap), and (b) **put the reliability controls in your own hands** and freeze the model so a result is re-runnable for an inspector. That is a *path* advantage, not a claim that a small model is smarter.

3 · This is a design, honestly labelled — not a demonstrated run. The walkthrough and worked example below are **illustrative of the intended pipeline**; no OQ validation run has been shown for this specific email→KB task. **Capability is established only by your own OQ test set, not by a model card.** If no small model passes your bar for a sub-task, that sub-task is *Beyond* on-device SLM — keep it deterministic SAS/R or escalate; do not ship a failing model.

Why Copilot-free, scoped fairly

The case rests on a real, cited failure *plus* two structural facts — used in proportion:

- **A cited reliability failure, scoped honestly.** On the Premium licence the org already pays for, the M365 Copilot Analyst Agent — on a real SDTM→ADaM→TLF QC task — shipped **empty “FINAL” files**, stamped work **“submission-ready”** and walked it back, **couldn’t keep a number still** (38%→34%→100%), **fabricated** an audit appendix, scrubbed its own failure summaries, and finally conceded **“you shouldn’t [use this] — for the workflow you just ran.”** See the [Field Evidence Exhibit](#). Read fairly: that is *one* high-stakes, execution-heavy QC session on one product surface — several failures are sandbox/platform limits, not model intelligence. It does **not** prove “Copilot never works,” and it is **not** the ops-KB task. It is enough to keep the AI step off the cloud for anything you want to *reproduce*.
- **You cannot freeze the model.** Microsoft versions the underlying GPT, so there is *no reproducible validation artifact*; the model is non-pinnable and Microsoft’s own line is that Copilot is “best suited for scenarios where deterministic accuracy is not required.” ([feasibility re-assessment](#))
- **You cannot keep the data on-prem.** The cloud GPT step is a different service surface; for genuinely de-identified ops content inside the tenant boundary that is usually acceptable — the zero-egress argument only really bites for unblinded/participant-level data, which this KB should never contain.

Below: the architecture, the build, the prompts, the governance, and the honest IT-readiness contrast. The parallel ships in the same package — choose the path that fits your tenant and your QA posture.

Watch it work — the Copilot-free pipeline

Illustrative of the intended design — not a captured production run. The video and screens below reuse the existing on-device-triage walkthrough, reframed to the email→KB scenario. No small model has yet been OQ-validated on this specific extraction task in your environment.

One study-operations email, from the inbox to a shared, queryable knowledge base, with the AI step running entirely on a local model — the guardrails shown *blocking*; operations content only, never a number.

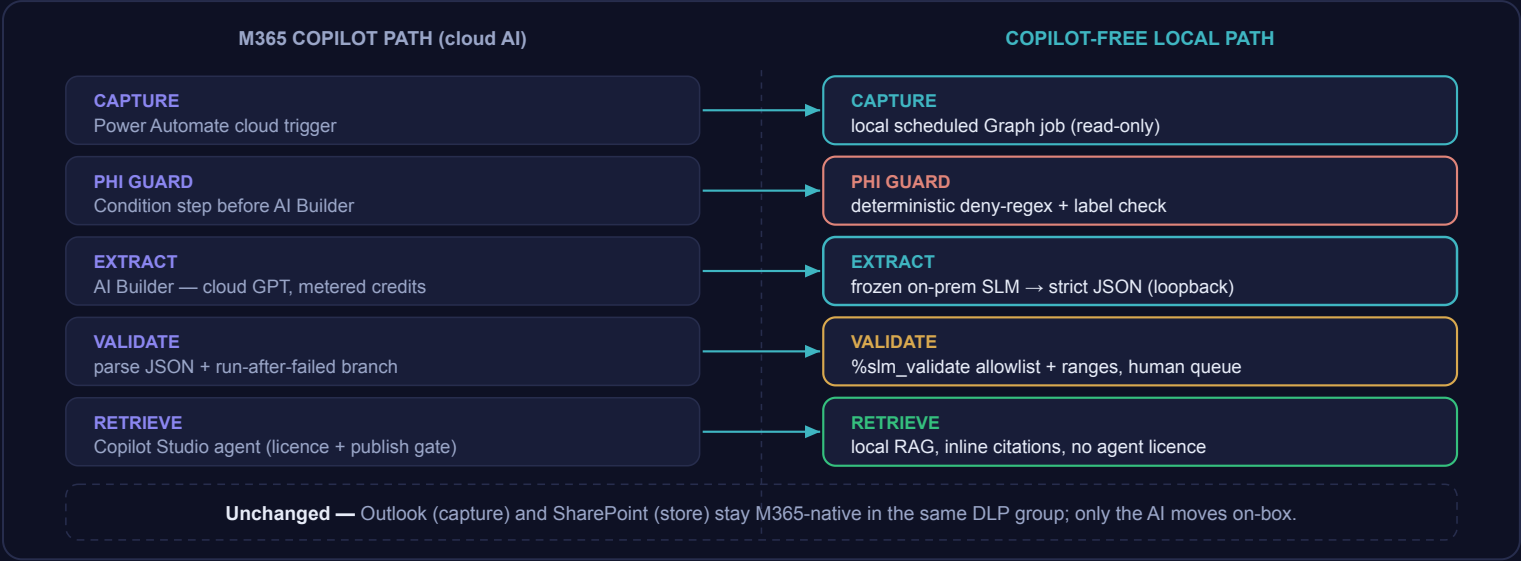
▶ [See it work — on-device, end to end](#)

↓ [Picture guide \(PDF\)](#)

The scenario mirrors the Copilot version: a CRO bioanalytical-assay re-validation email that moves the data cut. The local capture job's PHI guard drops a Restricted email to the human queue *before the model sees it*; an ops email is classified to strict JSON, validated, human-approved, and filed; and the local RAG answers a plain-language question with a citation back to the source email — refusing anything clinical.

Architecture — what replaces each cloud piece

Same shape as the Copilot pipeline; every cloud-AI component is replaced by a local, owned, frozen one. Outlook and SharePoint stay M365-native — only the *AI* moves on-box.



Each cloud-AI component on the left is swapped for a **local, owned, frozen** one on the right — capture trigger → Graph job, AI Builder → frozen SLM, the Copilot agent → local RAG. **Outlook and SharePoint do not move**, so the integration stays in one DLP group.

Step	M365 Copilot path	Copilot-free local path
Capture	Power Automate “When a new email arrives” cloud trigger on the tagged category	Same Outlook category; a local scheduled job (SAS/R or Python) on a service account polls the category via Microsoft Graph (read-only). No Power Automate, no cloud trigger.
PHI guard	A Condition step terminates on a Restricted/PHI label before AI Builder	The job checks the Graph sensitivity label + a deterministic deny-regex first and drops Restricted/unblinded/patient-level mail to the human queue — before the model sees it.
Extract	AI Builder “Run a prompt” (cloud GPT), metered by AI-Builder credits	A frozen on-prem SLM on loopback does schema-constrained summarise+classify to strict JSON (the LOCALMIND <code>%slm_classify</code>). Zero per-call cost, no credit clock, no egress.
Validate	Parse JSON + a run-after-failed branch to a human Teams message	<code>%slm_validate</code> checks the <i>enumerated</i> fields against a hard-coded allowlist + numeric ranges and fails loud ; anything malformed/off-list/uncertain routes to a human queue , never the list.
Store	SharePoint “Create item” on the Study Knowledge Base list	The job writes the <i>validated</i> row to the same SharePoint list via Graph (Outlook + SharePoint only — same DLP group), stamping the model+quant+digest+seed provenance for the audit line. A local CSV/SQLite mirror is the offline fallback store.

Step	M365 Copilot path	Copilot-free local path
Retrieve	A Copilot Studio agent grounded on the site answers with citations (needs Work IQ + a Copilot licence + the agent-publish control)	A local RAG over the KB: a local embedding model indexes the rows; the same frozen SLM answers with inline citations back to the source email. Native SharePoint search is the zero-AI fallback. No agent licensing.
Digest	A scheduled flow runs an AI-Builder digest prompt and writes a Weekly Digest page	The same local job: SAS/R selects and enumerates the week's rows (every Decision/Action/Risk/Milestone is a row the model cannot drop); the SLM may only <i>re-phrase</i> per-row text. One cron entry.

Honest boundary. The “pull the cable” demonstration proves the *AI/inference* step is offline — it does **not** mean the pipeline is cloud-free. Capture (Graph read) and Store (SharePoint write) still touch your M365 tenant, and the Graph permissions need the same tenant-admin / conditional-access review any tenant integration does.

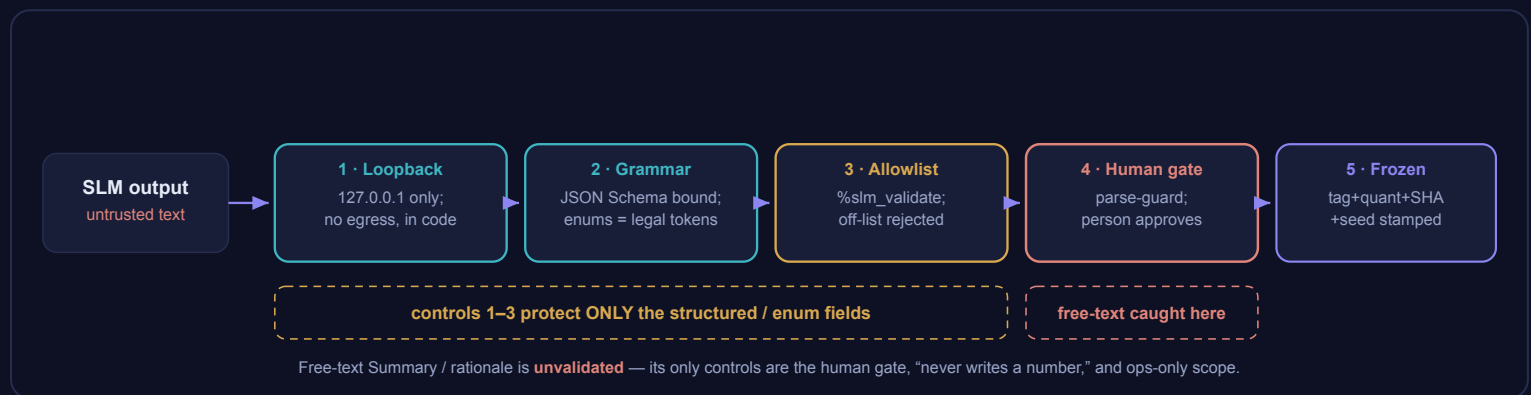
Build · 1–2. Capture, then local extract

1 · The capture job (replaces the cloud trigger)

An Outlook rule tags study-ops mail to a category. A service-account job — SAS/R or Python on the managed workstation — polls that category via Microsoft Graph (read-only) on a cron / Task-Scheduler cadence. **The PHI guard runs first:** the job checks the Graph sensitivity label and a deterministic deny-regex; anything Restricted/unblinded/patient-level is dropped to a human queue and *never* reaches the model. Idempotent upsert-by-key on `internetMessageId` means re-runs never duplicate rows.

2 · The local SLM extract (replaces AI Builder)

This is the core Copilot-free substitution. The frozen on-prem model summarises and classifies the email to **strict JSON** through the LOCALMIND `%slm_*` library, with five controls:



The model is treated as **untrusted text** behind five stacked controls. Controls 1–3 (loopback, grammar, allowlist) protect **only the structured / enum fields**; the free-text summary is unvalidated and is caught **only** by the human gate, the no-number rule, and ops-only scope.

- **Loopback-asserted egress boundary** — `%slm_init` refuses any non-127.0.0.1 endpoint; “nothing leaves the box” is enforced in code (demonstrable by pulling the cable), not by policy.
- **Grammar-constrained decoding** — a full JSON Schema is bound as the decode-time grammar, so the JSON is guaranteed parseable and enum fields can only emit allowed tokens (not merely “requested”).
- **The allowlist validator** — `%slm_validate` checks each *enumerated* field against a hard-coded allowlist and numerics against a range, and rejects off-list/out-of-range output. The model is treated as **untrusted text**.
- **The human-gated parse-guard** — anything malformed, off-allowlist, out-of-range, or low-confidence routes to a person; SAS/R writes a row only after a human approves.
- **Frozen & stamped** — model tag + quantization + SHA-256 + temperature 0 + fixed seed + runtime + hardware are pinned and stamped on every row for the audit line.

The most important honesty point on this page. The allowlist + grammar controls protect **only the structured classification fields** (the category/owner/severity *enums* and the numerics). The free-text fields — the per-row *Summary*, any *rationale* — are **unvalidated**: the model can still write a fluent summary that misstates the email. Their only real controls are the **human gate**, the fact that the model **never writes a number**, and ops-only scope. “Belt-and-suspenders validation” does *not* mean the prose is checked.

Build · 3–5. Store, ask the KB, weekly digest

3 · Store the validated row

The job writes the human-approved row to the same SharePoint *Study Knowledge Base* list via Graph — Outlook + SharePoint only, so it sits in one DLP group with no AI connector to cross-group-block. Provenance columns (model, quant, digest, seed, temperature, reviewer, timestamp) make each AI-assisted item attributable and re-runnable. A local CSV/SQLite mirror is the offline fallback if SharePoint write is unavailable.

4 · Ask the KB — local RAG (replaces the Copilot agent)

A local embedding model indexes each KB row/digest page into a local vector index; a query retrieves the top passages and the same frozen SLM answers **with inline citations** back to the source email subject + date.

RAG makes errors catchable, not answers correct. Retrieval reduces ungrounded claims but does not prevent a small model from mis-reading a passage or attributing a claim to the wrong chunk — the very failure mode the Field Evidence documents (confident text that doesn't match the artifact). So: every answer must **surface its citations inline** for a human to spot-check, an un-checked RAG answer is not trustworthy, and the “refuse clinical questions” behaviour is paired with the *same deterministic deny-regex pre-filter* used at capture — you do not rely on the model to refuse. Also note: the embedding model is a **second** un-pinned model with its own licence, provenance, freeze, and retrieval-quality bar — it must be governed alongside the generation model, because retrieval quality gates the whole answer.

5 · The weekly digest — deterministic-first

A model writing its own weekly summary is exactly the integrity failure the Field Evidence shows (self-summaries that front-load green checks and bury conceded failures). So the digest is **deterministic-first**:

- **SAS/R selects and enumerates** the week's rows — every Decision, Action, Risk, and Milestone is a row the model *cannot* drop.
- The SLM may only **re-phrase per-row text**; it never decides what to include or exclude.
- The human reviewer **reconciles the digest against the deterministic row count** — an omission is the thing a human rubber-stamping plausible prose will otherwise miss.

Prompt library — a starting point, not a drop-in

The extraction and digest prompts and JSON schemas carry over from the — the schema/grammar constraint transfers cleanly. The *prose behaviour* does not.

Do not assume verbatim transfer. A small model is more prompt- and format-sensitive than cloud GPT; a prompt tuned for a frontier model will *not* behave identically on an 8B Q4 model. Each prompt is a **starting point that must be re-tuned and re-validated** on your specific pinned model via the OQ test set. The schema/grammar carries over; the wording behaviour is re-earned per model.

- **Email-extraction prompt** → strict JSON: `study, category, headline, summary, owner, dueDate`; words only; self-redact to `category:"Other", summary:"REDACTED"` on any patient-level content. Grammar-enforced enums; `%slm_validate` on the structured fields.
- **Weekly-digest prompt** → re-phrase the four sections from the *already-enumerated* rows; cite the source headline per bullet; an explicit “do not infer clinical results” guard. (Selection is SAS/R’s, never the model’s.)
- **Ask-the-KB (RAG)** → answer only from the retrieved passages; cite each; refuse clinical/patient-level. Paired with the deterministic deny-regex pre-filter.

Field notes that will bite you (from the LOCALMIND README): turn streaming off for parsing; avoid double-parse; watch `num_ctx` truncation; quantization is part of the model identity; determinism is per-box.

Governance, guardrails & the honest reliability story

The one rule, again. Ops/language layer only. The SLM classifies, extracts, routes, and drafts short grounded text — it **never produces a number, never reconciles, never decides**. Every count, check, and statistic is SAS/R’s and the validated engines’; a qualified human signs. The KB is informational ops support, not a source of record.

What actually makes a small local model safe enough

Not its being local — the discipline around it. The controls, and *exactly* what each does and does not cover:

PROTECTS		DOES NOT PROTECT
Grammar decode	✓ parseable JSON, legal enum tokens	✗ truth of any free-text field
%slm_validate	✓ enum + numeric fields	✗ summaries, rationales, digest, RAG
RAG + citations	✓ makes claims catchable by a human	✗ correctness — small models mis-cite
Deny-regex	✓ Restricted/PHI never reaches model	— this is the real gate, not model "refusal"
Human gate	✓ nothing reaches record without a person	✗ can rubber-stamp — reconcile row counts
Frozen tuple	✓ run-to-run reproducible, attributable	✗ byte-exactness across hardware

What each control **does** and **does not** cover. The recurring uncovered seam is the same one: **free-text prose** (summaries, rationales, RAG answers, the digest) is only ever caught by a human — the validators never touch it.

Control	Protects	Does NOT protect
Grammar-constrained decoding	JSON is parseable; enums are legal tokens	The truth of any free-text field
%slm_validate allowlist + ranges	The structured category/owner/severity enums + numerics	Summaries, rationales, RAG answers, the digest
RAG grounding + inline citations	Makes ungrounded claims <i>catchable</i> by a human	Does not make the answer <i>correct</i> ; small models still mis-cite
Deterministic deny-regex (capture & ask)	Restricted/PHI never reaches the model	— (this is the real gate, not the model’s “refusal”)
Human-gated parse-guard	Nothing reaches the record without a person	A human can still rubber-stamp plausible prose — reconcile against row counts
Frozen tuple + Part 11 audit line	Run-to-run reproducibility on the pinned box; attributable, re-runnable	Byte-exactness across hardware (floating-point non-associativity)

Be fair to Copilot. For the de-identified ops/knowledge/drafting layer, M365 Copilot is genuinely capable (the feasibility re-assessment rates 19/75 fully feasible, most of the rest buildable; SOP RAG Q&A, minutes and status drafting, and action-item extraction are clean wins). And the local 8B model hallucinates *more* than the cloud GPT — so instability is a risk the local path shares for its free-text steps. The local advantage here is the **IT/reproducibility path and ownership of the controls**, not superior model reliability.

IT readiness — what it avoids, and what it newly requires

The honest contrast. Hand IT the [SLM Enablement Runbook](#). The local path *trades* Copilot's frictions for a different set — it is not zero-friction.



The local path is **not zero-friction**: it drops Copilot's DLP / credit / licensing / egress / no-freeze frictions but takes on a **qualified instrument to maintain** — and several requirements (QA refusing to OQ an LLM, weight provenance, the re-validation treadmill, bus-factor of one) are **possibly blocking**.

What the local path avoids vs the Copilot path

- **The DLP-connector cross-group block** — only Outlook + SharePoint connectors, same data group; no AI-Builder “Run a prompt” connector to land in a different group and suspend the flow.
- **The 01-Nov-2026 AI-Builder credit retirement** + the subsequent Copilot-Credits PAYG meter — no metered cloud GPT step to provision or fund.
- **Copilot Studio per-use licensing** + the tenant agent-publish control + the Work IQ / M365-Copilot-licence dependency — the local RAG needs none of it.
- **Cloud egress for the AI step** + a separate BAA surface — the model runs on loopback; the control is provable by pulling the cable.
- **The no-model-freeze reproducibility gap** — the local model is pinned (tag+quant+SHA-256+temp 0+seed) and registered as a re-runnable artifact.

What it newly requires — honestly, including the hard parts

- **One managed workstation/VM** (CPU-first: ~6 GB resident for an 8B Q4, 16 GB+ RAM alongside SAS/R; add one consumer 12–16 GB GPU only if a real queue demands throughput), with full-disk encryption + endpoint management — it now holds study text and runs an inference service.
- **A signed, version-pinned runtime** (Ollama / llama.cpp) from the internal mirror, and an **air-gapped, checksummed model transfer** — nothing pulled live on production. Clears “we don’t allow unknown binaries.”
- **Legal sign-off on the specific model and its exact size variant licence** — prefer Apache-2.0 / MIT (IBM Granite 4.1 8B, Phi-4-mini, Qwen3, Gemma 4); Llama 3.3/3.2 is under the Llama Community Licence (read it). Note across releases (June 2026): Qwen consolidated to uniform Apache-2.0 in Qwen3,

and Gemma moved from a custom licence (Gemma 2/3) to Apache-2.0 in Gemma 4 — verify the exact release and size you pull.

- **A service/group account** with Graph read on the mailbox + SharePoint write — which needs the *same* tenant-admin / conditional-access review any tenant integration needs.
- **A named owner + a documented patch cadence**; the model is under a change-controlled freeze registry.

The hard, possibly-blocking parts — do not soft-pedal these.

- **Qualifying a non-deterministic generative component is unsettled ground.** There is no accepted industry template for “OQ a small language model”: defining the fixed test set, the pass/fail bar for free-text, and the re-run stability criteria is substantial *novel* validation work, and a QA group may simply **refuse to qualify any LLM** in a GxP-adjacent path. Treat this as potentially blocking, not routine.
- **Weight provenance is a real gap.** SHA-256 proves the file didn’t change in transit — not that the source (often a third-party community quantizer) is trustworthy or that the quant is unmodified. For a regulated environment, weight provenance/attestation is unsolved.
- **The re-validation treadmill.** Every model/quant/runtime/OS-math-library/hardware change is a *controlled change requiring re-validation* (re-run the OQ set). On a multi-year trial, on one workstation that one named person owns, that is a recurring re-qualification burden and a bus-factor of one.
- **“No credit clock” is not “no cost.”** You have replaced a metered cloud entitlement with a **qualified instrument you must maintain** — named owner, patch cadence, OQ upkeep, endpoint hardening — like Phoenix or Pinnacle 21. The honest comparison is *metered-cloud-entitlement vs one-time-provisioning + ongoing CSV burden*, not vs “free.”
- **“Arguably more tractable” is org-dependent.** In a hardened shop, approving a new local inference binary + a deliberately cable-out box can be a multi-month committee process — sometimes harder than turning on an already-licensed M365 feature.

Worked example — a week on CP-101 (illustrative)

Illustrative of the intended design, not a captured run. It shows the discipline, not a benchmark.

1. **Mon.** Nine study-ops emails carry the category. The capture job pulls them; the deny-regex drops one Restricted vendor email to the human queue. Eight are classified to strict JSON, validated, and presented for approval; the reviewer edits one owner and approves — eight rows filed, with provenance.
2. **Tue.** “What moved the data cut, and who owns it?” The local RAG retrieves the assay-re-val row and answers with an inline citation to the source email; the reviewer clicks through and confirms the cited passage actually supports it.
3. **Thu.** A malformed email yields off-allowlist JSON; `%slm_validate` rejects it and routes it to the human queue — it never reaches the list.
4. **Fri.** SAS/R enumerates the week’s rows (3 decisions, 4 actions, 2 risks, 1 milestone); the SLM re-phrases each; the reviewer reconciles the digest against that count — nothing dropped — and the Weekly Digest page posts.

No number in any of this came from the model. The data cut date, any counts, and every reported value remain the validated pipeline’s and a human’s.

Maintenance & FAQ

Monthly upkeep (~15–30 min, plus change control)

- Review the list + permissions; prune stale items; confirm provenance stamps are populating.
- Skim the job's run log and the human-queue volume; if categories drift, *re-tune and re-OQ* the extraction prompt on the pinned model — a prompt change is a controlled change.
- Confirm the model, quant, runtime, and hardware are unchanged vs the freeze registry. Any change triggers re-validation.

FAQ

Is it reliable? Only with the guardrails — and even then a small model hallucinates, more than a frontier one. It is safe enough *for the ops layer* because it never produces a number, its structured output is validated, its free-text is human-checked against citations, and anything uncertain goes to a person. It is not a source of truth.

Does it cost extra? No per-call meter and no Copilot credits — but it is not free: it is a qualified instrument with a real, recurring validation + ownership cost (a named owner, OQ upkeep, re-validation on any change, endpoint hardening).

Can it read PHI? No — by policy and by the deterministic deny-regex at capture, restricted mail never reaches the model; and the model runs on loopback so even a guard miss cannot egress. Capture and store still touch the tenant, though.

Why not just use Copilot? For this task Copilot is capable, but you cannot freeze its model (no reproducible artifact) and it carries the DLP/credit/licensing/agent-publish frictions. Go local when reproducibility and control matter more than M365-native convenience — not because a small model is smarter.

Won't it go stale? Like any KB — unless someone owns it. Assign a steward, add an effective-date/superseded field, set a monthly review, and keep the “not a source of record” disclaimer visible.

What about the “ships-Monday” no-AI fallback? Outlook + SharePoint + native search + a light template (no model at all) captures a surprising share of the value with zero validation burden. If that is enough, it is a fair reason to *defer* standing up and qualifying a local LLM — be honest that the model's marginal value must clear its own maintenance cost.